

SmartAttend — Module Breakdown

Detailed technology, task, and role-fit reference for assigning module ownership among team members

This document expands each of the four SmartAttend modules into concrete detail: the exact technologies involved, the specific tasks the owner will perform day to day, the skills/background that make someone a good fit, and what a finished deliverable looks like. All four modules involve genuine, demoable coding work — documentation and cross-module testing are treated as light, shared responsibilities rather than a standalone module, since AI assistance already handles most of the heavy drafting. Use this to match modules to team members based on real interest and existing skills, not just availability.

Module 1: Face Recognition & Liveness (Computer Vision / ML)

Owns everything that happens between a face appearing on camera and a confidence-scored, spoof-checked identity being handed to the backend.

What this module is responsible for

This is the core AI component of SmartAttend. The person owning this module builds the pipeline that takes a raw camera frame and turns it into a decision: who is this, how confident is the system, and is it a live person or a spoof attempt (photo/video held up to the camera).

Technologies used

Category	Specific Tools / Technologies
Language	Python
Face Detection	OpenCV (Haar Cascade / DNN module) or MTCNN for locating faces in a frame
Face Recognition	FaceNet or ArcFace embeddings, accessed via the DeepFace or InsightFace Python library (avoids training a model from scratch)
Liveness Detection	MediaPipe Face Mesh for eye-landmark tracking (blink detection via Eye Aspect Ratio), or a simple CNN-based texture classifier as a fallback
Similarity / Confidence	Cosine similarity or Euclidean distance between embeddings, scikit-learn for threshold tuning and evaluation metrics
Data Handling	NumPy for array/embedding operations, Pandas for organizing dataset records
Hardware Interface	ESP32-CAM or standard webcam, OpenCV VideoCapture for frame grabbing

What the person will actually do

- Collect and organize the face dataset: 10–15 reference images per enrolled student, varying angle and lighting.
- Preprocess images (face cropping, alignment, resizing) before generating embeddings.

- Generate and store face embeddings for every enrolled student using a pretrained recognition model.
- Write the matching function: compare a live-captured face embedding against stored embeddings and compute a confidence score.
- Implement threshold logic: decide the exact confidence cutoff above which a match auto-marks attendance, and below which it routes to manual verification.
- Build the liveness check: detect eye blinks or analyze texture patterns to reject photos/videos held up to the camera.
- Test recognition accuracy across different lighting conditions and angles, and tune thresholds based on real results.
- Package the whole pipeline as a function or small API the backend module can call with a captured frame and get back a structured result (student ID, confidence score, liveness pass/fail).

Good fit if you...

... are comfortable with Python and enjoy working with images/data rather than UI or servers; have used OpenCV or done any prior computer vision work; like the idea of tuning a model until numbers improve; don't mind the "it doesn't work and I don't know why yet" debugging cycle that comes with ML work.

Deliverable at end of module

A working recognition + liveness pipeline that takes an image/frame as input and returns identity, confidence score, and liveness result — plus a short accuracy report (true positive / false positive / false reject rates on the test dataset).

Module 2: Core Backend & Database

Owns the server foundation and the database — the plumbing that Modules 1, 3, and 4 all build on top of.

What this module is responsible for

This module is the system's backbone. It receives recognition results, writes attendance records, sets up authentication, and serves core data to the frontend. It does not own the appeal workflow, notifications, or predictive features — those belong to Module 4, which builds on top of what this module provides.

Technologies used

Category	Specific Tools / Technologies
Language / Framework	Python, Flask (routing) with Jinja2 templating for server-rendered pages
Database	Firebase Firestore or AWS DynamoDB/RDS — full schema matching the ER diagram (Student, Faculty, Course, Enrollment, AttendanceRecord, Appeal tables)
Authentication	Flask-JWT-Extended (JWT-based tokens) with role-based access control (student / faculty / admin)
Cloud Storage	Firebase Storage or AWS S3 for storing reference face images/embeddings
API Testing	Postman — for building and documenting the core API collection
Version Control	Git/GitHub for source control and collaboration with the other modules

What the person will actually do

- Set up the Flask project structure and core routes (attendance logging, core dashboard data, login).
- Design and implement the full cloud database schema matching the ER diagram, including all foreign key relationships (Module 4 will build appeal-specific logic on top of the Appeal table this module creates).
- Build the endpoint that receives a result from the Recognition module (student ID + confidence + liveness) and writes an AttendanceRecord.
- Implement authentication and role-based access so students, faculty, and admins see only what they should.
- Build core data endpoints for the dashboard and portal: attendance lists, per-student history, basic aggregation (e.g. attendance percentage by subject).
- Test every core endpoint in Postman, handle error cases (invalid input, low confidence, missing records) gracefully.
- Coordinate with Module 4 on the exact API contract they'll extend for appeals, notifications, and predictions.

Good fit if you...

... are comfortable with Python and like building things that connect other pieces together; have any prior experience with Flask, REST APIs, or databases; enjoy thinking through data models and edge cases; are fine reading cloud

service documentation (Firebase/AWS) to wire things up.

Deliverable at end of module

A running Flask backend with core REST API endpoints, a fully populated cloud database matching the ER diagram, working authentication, and a Postman collection demonstrating every core endpoint works.

Module 3: Frontend (Faculty Dashboard & Student Portal — Core UI)

Owns the core interfaces users see day to day — turning backend data into two usable, real-time dashboards.

What this module is responsible for

This module builds the core shell and everyday views of SmartAttend's two web interfaces: the faculty dashboard (attendance table, analytics, trends) and the student portal (attendance status, history). The appeal submission/review screens and the “What If?” planner UI are owned by Module 4, as part of that feature's full vertical slice — this module focuses on the core, always-visible views.

Technologies used

Category	Specific Tools / Technologies
Core	HTML, CSS, vanilla JavaScript
Templating	Jinja2 (server-rendered pages, integrates directly with the Flask backend)
Styling	Bootstrap (via CDN) for layout and responsive design without a build step
Charts	Chart.js for attendance trend graphs and visual analytics on the faculty dashboard
Live Updates	AJAX / fetch() polling every few seconds to keep the dashboard and portal near real-time without needing WebSockets
Design Reference	Wireframes/mockups sketched before building, to agree on layout before writing code

What the person will actually do

- Design and build the faculty dashboard core view: live attendance table, confidence breakdown per student, and trend charts.
- Design and build the student portal core view: current attendance percentage and subject-wise breakdown.
- Wire both core views up to Module 2's REST APIs using fetch/AJAX calls, handling loading and error states.
- Implement near real-time updates on the dashboard via polling, so new attendance events appear without a manual refresh.
- Handle responsive design so pages are usable on both desktop and mobile browsers.
- Apply visual polish and establish the shared design system (colors, spacing, components) that Module 4 will reuse when building the appeal and planner screens, so the whole product feels consistent.
- Coordinate with Module 4 on where their appeal/planner screens plug into the navigation and layout.

Good fit if you...

... enjoy HTML/CSS/JS and care about how things look and feel to use; like seeing visible, clickable progress rather than backend logic; are comfortable calling APIs from JavaScript (fetch); have an eye for clean, uncluttered layout.

Deliverable at end of module

A functional faculty dashboard and student portal core view, both connected to live backend data, demonstrating real-time attendance updates and a consistent design system for Module 4 to extend.

Module 4: Appeal Workflow, Notifications & Predictive Features

Owns a complete feature top to bottom — backend logic AND the UI for it — rather than one horizontal layer.

What this module is responsible for

This module is a full vertical slice: it takes the appeal workflow, absence notifications, defaulter prediction, and the “What If?” planner all the way from backend logic to the screens students and faculty actually use. It builds on top of Module 2's database and auth, and plugs its screens into Module 3's design system — but the feature itself, end to end, belongs entirely to this module. This is intentionally structured as full-stack work so it carries equal technical weight to Modules 1–3, not a documentation or coordination role.

The “What If?” planner specifically is built as an **agentic AI feature**, not a fixed formula. Instead of hardcoding “percentage = attended / total,” the student asks a free-form question (“Can I skip Friday's DBMS lecture and still stay above 75%?”) and an LLM agent (via the Claude API or OpenAI API, using function/tool calling) decides which backend functions to call to answer it — pulling real attendance data, running the calculation, and replying in plain language. This is what makes the planner genuinely “smart” rather than a calculator with a chat box on top: the reasoning about which data to pull and how to frame the answer is done by the model, not by a fixed if/else path.

Technologies used

Category	Specific Tools / Technologies
Backend	Python, Flask — new routes built on Module 2's existing app and database connection
Database	The Appeal table (from the shared ER diagram/schema) plus queries against AttendanceRecord for prediction logic
Notifications	Firebase Cloud Messaging (push) or Twilio/SMTP (SMS/email) for same-day absence alerts
Defaulter Alerts	Python — straightforward statistical calculation over historical attendance records (this stays simple; it's a background alert, not something a student converses with)
Agentic AI Planner	Claude API (or OpenAI API) with function/tool calling — the LLM acts as an agent that calls real backend functions (e.g. get_attendance_history, calculate_projection) and reasons over the results to answer free-form questions
Frontend	HTML, CSS, vanilla JavaScript, reusing Module 3's Bootstrap/Chart.js design system
API Testing	Postman — for the appeal, notification, and planner endpoints specifically

What the person will actually do

- Implement the Appeal table CRUD operations in the backend: create on submission, update status on review, correct the linked attendance record if approved.
- Build the student-facing appeal submission form and the faculty-facing appeal review interface (approve/reject), using Module 3's design system.

- Implement the same-day absence notification trigger and integrate it with FCM/Twilio/SMTP.
- Write the defaulter prediction logic: analyze attendance patterns per student and flag those at risk of falling below the required threshold, then add a small alert panel to the faculty dashboard to display it.
- Define the backend “tools” the planner agent can call — e.g. `get_attendance_history(student_id, course_id)` and `calculate_projection(current_data, hypothetical_absences)` — as plain Python functions with clear inputs/outputs.
- Integrate the Claude API (or OpenAI API) with function/tool calling so the LLM agent can call those tools, reason over the returned data, and generate a natural-language answer to the student's question.
- Write the system prompt and conversation handling so the agent stays scoped to attendance questions and doesn't hallucinate numbers — it must always call the real tool for data rather than guessing.
- Build the planner's chat-style input/output UI on the student portal, including a fallback message if the API call fails or times out.
- Test the agent against a range of phrasings (“can I miss two more classes,” “what if I skip Monday,” etc.) to confirm it reliably calls the right tool and gives a correct, consistent answer.
- Test the entire appeal → review → resolution flow, the notification trigger, and the planner end to end.
- Coordinate with Module 2 on the API contract and with Module 3 on where these screens fit into navigation and layout.

Good fit if you...

... like owning one complete feature rather than one horizontal layer; are comfortable jumping between backend (Flask/Python) and frontend (HTML/JS) on the same task; are curious about LLM APIs and function/tool calling, even without prior experience — this is a good module to learn agentic AI concepts hands-on; enjoy prompt iteration and testing edge cases rather than one-shot deterministic logic; don't mind that this module depends on Module 2's database being ready first, which means light coordination early on.

Deliverable at end of module

A fully working appeal submission → review → resolution flow, working same-day absence notifications, working defaulter alerts on the faculty dashboard, and a working agentic “What If?” planner that correctly answers free-form attendance questions by calling real backend data — each with its own UI, demonstrable end to end.

Testing & Documentation (Shared Responsibility)

Testing and documentation are intentionally not a standalone module. Each module owner tests their own piece as part of finishing it — recognition accuracy for Module 1, API correctness for Module 2, UI/functional checks for Module 3, and end-to-end flow testing for Module 4. This is normal practice and means every team member has a real testing story to describe in Q&A, rather than one person owning testing for work they didn't build.

Documentation (SRS, architecture/ER diagrams, reports) is largely AI-assisted and drafted centrally, so it no longer justifies a full module on its own. Keeping it current and integrating Git contributions across the four modules is treated as a light, rotating responsibility — a few hours per person per review cycle, not a full-time role.

Deployment

Deployment is primarily owned by Module 2 (Backend & Database), since the Flask app is the piece being hosted and everything else runs through it — but every module owner should understand this plan, since their code ends up running on it.

Category	Specific Tools / Technologies
Backend Hosting	Render or Railway (free/hobby tier is enough for a classroom pilot) — hosts the Flask app, including Module 3's Jinja2-rendered pages and Module 4's routes, since they all live in the same app
Database & Storage	Firebase Firestore + Firebase Storage (or AWS equivalents) — already cloud-hosted by design, no separate deployment step needed
Frontend	No separate hosting required — served directly by Flask via Jinja2 templates, so the dashboard/portal deploy automatically with the backend
Secrets & Config	Environment variables on the hosting platform (API keys for Claude/OpenAI, Firebase credentials, JWT secret) — never committed to Git
Edge Device	ESP32-CAM or laptop webcam physically set up at the pilot classroom entry point, configured to call the deployed backend's API over Wi-Fi
Domain / Access	Platform-provided subdomain (e.g. smartattend.onrender.com) is sufficient for a pilot; no custom domain needed

Keep it simple for this project cycle: one deployed environment (no separate staging/production split), manual redeploy after pushing to the main branch is fine — a full CI/CD pipeline is not necessary to demonstrate a working pilot, and is better listed as future work than built under time pressure.

How to Use This to Decide

A simple way to assign these without conflict: each team member privately ranks all four modules 1–4 by interest (not perceived difficulty). Compare rankings as a group — in most teams, preferences naturally spread out because the four modules genuinely appeal to different strengths (ML/data vs. backend/systems vs. UI/design vs. full-stack feature ownership). Where two people want the same module, let the person with more directly relevant prior experience take it, and the other person can pair on it for the first few days before shifting to their second choice.

Note: Module 4 (Appeal Workflow, Notifications & Predictive Features) depends on Module 2's database and auth being in place first, and its screens plug into Module 3's design system — so a small amount of early coordination between these three is expected. This is normal for any real system and is itself a good story to tell the panel about how the team worked together.